ACM / ICPC

算法模板手册

Rejected | 2026-10-11

---

1 个章节 • 11 个条目 • 11 个代码块

使用约定

本手册各代码块独立选用；同名结构体、多个 main 和不同版本不要一起粘贴。片段按题意补齐输入输出与全局数组。

- 默认使用 GNU C++17；`__int128`、`bits/stdc++.h` 和 builtin 属于 GCC 扩展，个别片段另需 C++20/23。
- 常用别名：`using ll = long long; using db = double; using i128 = __int128_t; using pii = pair<int,int>;`。
- 下标和区间以各条目说明为准；多测清空边表、计数器、懒标记与答案，递归深树注意栈空间。
- 先估算时间与内存；乘法在计算前转宽类型，模逆先确认存在，哈希和浮点结果注意碰撞与误差。
- 原有题目解法、样例、重复版本全部保留；标为“片段/骨架”的代码须补齐依赖后使用。

如何添加算法

在对应一级章节下添加二级算法标题，可在代码前写“用法 / 注意 / 复杂度”。一个条目支持多个代码块、正文、公式和表格。

````
## 算法名称

**用法**：调用入口、下标和返回值。**注意**：适用条件。

```cpp
// 你的代码
```
````

运行 `python make.py` 生成 PDF；`python make.py --no-pdf` 仅生成 TeX。需要 XeLaTeX（MiKTeX / TeX Live），Python 无需第三方包。

版式参考：KACTL、Stanford ACM Notebook。

目录

1 不要忘记 **3**

1.1 土制 cph(仅 linux) 3
1.2 土制 cph(全平台) 3
1.3 随机数 3
1.4 对拍 3
1.5 解除 python 大数限制 3
1.6 常用函数 3
1.7 高精度 4
1.8 多进程评测 4
1.9 迭代器辅助函数 4
1.10 set 相关 4
1.11 vector 相关 5

1 不要忘记

1.1 土制 cph(仅 linux)

用法：保存为 Python 脚本，传入题号和源文件名；读取 samples-题号中的.in/.ans。注意：依赖 Linux timeout 与 g++；文件名不要含空格，先确认编译成功。

```
from os import system as e, listdir as l
from sys import argv
_, q, f = argv  # q: 题号, f: 文件名

if "py" in f:
  r = f"python3 {f}"
else:
  e(f"g++ -std=c++23 -O2 -Wall {f}.cpp -o {f}")
  r = "./" + f

#
d = "samples-" + q.capitalize()
for i in l(d):
  if not "in" in i: continue
  print(i)
  p = d + "/" + i[:-2]
  if e(f"timeout 2 {r}<{p}in>{p}out"): print("TLE or RE")
  else:
    with open(f"{p}out") as o, open(f"{p}ans") as a:
      print("AC" if o.read().split()==a.read().split() else "WA"
  )
```

1.2 土制 cph(全平台)

用法：同上，Python 分支传带.py 的文件名，C++ 分支传不含扩展名的文件名。注意：需可用的 Python/g++，非零退出码按 RE 处理；浮点比较须同时检查 token 数量。

```
from os import listdir as l, name as osname
from os.path import join as jp
from sys import argv as a, executable
import subprocess as s
_,q,f = a
if f.endswith(".py"):
    cmd = [executable, f]
else:
    exe = f + (".exe" if osname == "nt" else "")
    s.run(["g++", "-std=gnu++20", "-O2", "-Wall", f + ".cpp", "-
  o", exe], check=True)
    cmd = [jp(".", exe)]

d='samples-'+q.capitalize()
g,r,p,n='\033[32m','\033[31m','\033[35m','\033[0m'
for i in l(d):
    if not i.endswith('.in'): continue
    t=jp(d,i[:-2])
    print(i,'测评结果 ',end='',flush=1)
    try:
        k = s.run(cmd, timeout=2, stdin=open(f"{t}in"),
                  stdout=open(f"{t}out", 'w'), stderr=s.PIPE,
  text=1)
        if k.returncode:
            print(f'{r}RE{n}')
            if k.stderr: print(k.stderr)
            continue
        c = lambda o,a : o.read().split() == a.read().split()
        # c = lambda o,a : all(abs(float(x)-float(y))/max(1,abs(
  float(y)))<=1e-4
        #              for x,y in zip(o.read().split(),a.read()
  .split()))
        print(f'{g}AC{n}' if c(open(f"{t}out"),open(f"{t}ans"))
  else f'{r}WA{n}')
        if k.stderr:print(f'{r}{k.stderr}{n}')
    except s.TimeoutExpired as k:
        print(f'{p}TLE or RE{n}')
        if k.stderr: print(f'{r}{k.stderr}{n}')
```

1.3 随机数

用法：Rand(x,y) 返回闭区间 [x,y] 的均匀随机整数，要求 x<=y；shuffle 打乱序列。注意：对拍复现时固定种子。

```
mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().
  count());
int Rand(int x, int y) {
  return uniform_int_distribution<int>(x, y)(rng);
}
shuffle(a.begin(), a.end(), rng);
```

1.4 对拍

用法：依次运行生成器、暴力程序和待测程序，比较 A.out 与 A.ans。注意：先确认三个命令都成功；Windows 用 fc，Linux 用 diff，保留首个失败输入。

```
// checker.cpp
while (1) {
  system("A_gen > _data.in");  // linux 下换成 ./xxx
  system("A_bf < _data.in > A.ans");
  system("A < _data.in > A.out");
  if (system("fc A.out A.ans")) {  // linux 下换成 diff
    cout << "WA\n"; return 0;
  } else {
    cout << "AC\n";
  }
}
```

1.5 解除 python 大数限制

用法：Python 3.11+ 在转换超长十进制整数前设置；此处仍限制为 100005 位。注意：set_int_max_str_digits(0) 才是完全解除；旧版本需判断该函数是否存在。

```
import sys
sys.set_int_max_str_digits(100005)
```

1.6 常用函数

用法：read 读取 int，qpow(a,b) 求模幂，qpow(a) 求逆元。注意：快读假定输入完整；模运算传入 [0,MOD) 的数，逆元要求 MOD 为质数且 a 非零；乘积超 ll 时用 i128。

```
// 快读
inline int read()
{
  int x=0,f=1;char ch=getchar();
  while (ch<'0'||ch>'9'){if (ch=='-') f=-1;ch=getchar();}
  while (ch>='0'&&ch<='9'){x=x*10+ch-48;ch=getchar();}
  return x*f;
}
```

```

// 取模
const int MOD=998244353;
ll add(ll x, ll y) { return (x+y)%MOD; }
ll del(ll x, ll y) { return add(x, MOD-y); }
ll mul(ll x, ll y) { return (x*y)%MOD; }
ll qpow(ll a, ll b = MOD - 2) {
  ll res = 1;
  for (; b; b >>= 1) {
    if (b & 1) { res = mul(res, a); }
    a = mul(a, a);
  }
  return res;
}

// 计算一个整数的二进制表示中有多少个 1
__builtin_popcountll(i);
std::popcount((unsigned long long)i);  // c++20

// 计算一个整数的二进制表示中最高位的 1
__lg(i);  // i > 0; 按实参类型支持 int / long long
```

1.7 高精度

用法：先定义 __int128 的流运算符，再以 cin/cout 读写；fixed/setprecision 只控制浮点显示。**注意**：__int128 是固定 128 位整数，不是任意精度；本版不处理最小负数取反。

```

cout << fixed << setprecision(15);//输出精度

istream& operator>> (istream& is, __int128 &x)
{
  string s; is>>s;
  bool f=false;
  __int128 ans = 0;
  for (auto c: s) {
    if (c == '-') { f = true; }
    if (isdigit(c)) { ans = ans * 10 + (c-'0'); }
  }
  x = f? -ans: ans;
  return is;
}
ostream& operator<< (ostream& os, __int128 &x)
{
  string ans;
  function<void(__int128)> write = [&](__int128 x){
    if (x < 0) { ans += '-'; x=-x; }
    if (x > 9) { write(x/10); }
    ans += '0'+x%10;
  };
  write(x); return os << ans;
}
```

1.8 多进程评测

用法：此处仅为接收进程编号并读写数据的示例。**注意**：它不会自动启动多个进程；并发调度与每个进程的独立输入输出需另写。

```
#include <cstdlib>
#include <iostream>
using namespace std;
int main(int argc, char *argv[]) {
    int id = argc > 1 ? atoi(argv[1]) : 0;
    int v;
    cin >> v;
    cout << v << endl;
    cout.flush();
    return 0;
}
```

1.9 迭代器辅助函数

注意：不要递增 end()、递减 begin() 或解引用 end()；set 的迭代器不支持减法，distance 是线性时间。

```

函数  作用  是否修改原迭代器  set 上复杂度
std::next(it) 返回 it 后一个位置的迭代器  否  O(1)
std::next(it, n)  返回 it 后 n 步的迭代器 否  O(n)
std::prev(it) 返回 it 前一个位置的迭代器  否  O(1)
std::prev(it, n)  返回 it 前 n 步的迭代器 否  O(n)
std::advance(it, n) 把 it 本身移动 n 步 是  O(n)
std::distance(a, b) 计算 a 到 b 的步数  否  O(n)
++it / --it 前进一步 / 后退一步 是  O(1)
注意:
std::next、std::prev 返回的是新迭代器，原来的 it 不变。

std::advance 会直接改 it，没有返回值。

对 set 来说，std::distance 是 O(n)，不要在高频查询中使用。元素个数与 key 差值不同；要 O(log n) 查排名，请使用树状数组/平衡树。
```

1.10 set 相关

用法：优先用 s.lower_bound/upper_bound 进行对数查找。**注意**：比较器须严格弱序，不能用 <=；multiset.erase(x) 会删掉全部 x，只删一个时用 find 后 erase(it)。

```
// 默认：从小到大（std::less<int>）
set<int> s1;
// 从大到小（std::greater<int>）
set<int, greater<int>> s2;
// 自定义比较器（仿函数）
struct GreaterCmp {
    bool operator()(int a, int b) const {
        return a > b;   // 从大到小
    }
};
set<int, GreaterCmp> s3;
// 用 lambda 自定义比较器（注意：lambda 类型没有默认构造函数）
auto cmp = [](int a, int b) { return a > b; };
set<int, decltype(cmp)> s4(cmp);
// insert(value) 返回 pair<iterator, bool>
//   - first: 指向插入元素或已存在元素
//   - second: 是否插入成功
// insert(hint, value) 返回 iterator
// hint 正确时均摊 O(1)，否则 O(log n)
s1.insert(s1.end(), 20);
// 插入范围
vector<int> v = {5, 6, 7};
s1.insert(v.begin(), v.end());
// emplace 原地构造
s1.emplace(30);
// ------------------------------------------------------------
// 3. 删除
// ------------------------------------------------------------
```

```cpp
// 按 key 删除，返回删除个数（set 中为 0 或 1）
size_t cnt = s1.erase(10);
auto it2 = s1.find(20);
if (it2 != s1.end()) {
    it2 = s1.erase(it2);   // it2 现在指向下一个元素
}

// 删除范围 [first, last)
// s1.erase(s1.begin(), s1.end());
// 清空
// s1.clear();
// ------------------------------------------------------------
// 4. 查找
// ------------------------------------------------------------
// find(key) 找到返回迭代器，否则返回 end()
auto it3 = s1.find(5);
if (it3 != s1.end()) {
    // 找到了
}
// count(key) set 中返回 0 或 1
if (s1.count(5)) {
    // 存在
}

// lower_bound(key) / upper_bound(key)
// 默认 less 时：
//   lower_bound: 第一个 >= key
//   upper_bound: 第一个 > key
// 使用 greater 时语义相反（降序）：
//   lower_bound: 第一个 <= key
//   upper_bound: 第一个 < key
auto lb = s1.lower_bound(5);
auto ub = s1.upper_bound(5);

// equal_range 返回 pair(lower_bound, upper_bound)
auto [lo, hi] = s1.equal_range(5);
// ------------------------------------------------------------
// 5. 迭代器与容量
// ------------------------------------------------------------
// s1.begin(), s1.end()
// s1.cbegin(), s1.cend()
// s1.rbegin(), s1.rend()
// s1.size(), s1.empty()
// ------------------------------------------------------------
```

1.11 vector 相关

注意：min/max_element 要求非空；unique 只去相邻重复，须先排序；reserve 不改变 size，resize 会；扩容后旧迭代器、引用可能失效。

```cpp
// 多维 vector，需要 c++17，c++14 只能老老实实写 vector<vector<int>>
vector a(n+1, vector (n+1, vector<int>(n+1)));

// 最值
int mx = *max_element(vec.begin(), vec.end(), cmp);
int mn = *min_element(vec.begin(), vec.end(), cmp);

// 去重与离散化
for (int i = 1; i <= n; ++i) { vec.push_back(a[i]); }
sort(vec.begin(), vec.end());
vec.erase(unique(vec.begin(), vec.end()), vec.end());
for (int i = 1; i <= n; ++i) {
  idx[i]=lower_bound(vec.begin(), vec.end(), a[i])-vec.begin()+1;  // 下标从1开始
}

// 前缀和
vector <ll> a(n), s(n);
partial_sum(a.begin(), a.end(), s.begin());
ll sum = accumulate(a.begin(), a.end(), 0LL);

// 填充 0~n-1，令 a[i]=i
iota(a.begin(), a.end(), 0);
```